МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
ИТМО»

Лабораторная работа №3-4
по дисциплине “Программирование”

Студент:
Антонова Анна Игоревна
Преподаватель:
Бобрусь Александр Владимирович

Санкт-Петербург
2024

Вариант №24.

Задание.

Лабораторная работа #3-4

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Введите вариант: 24

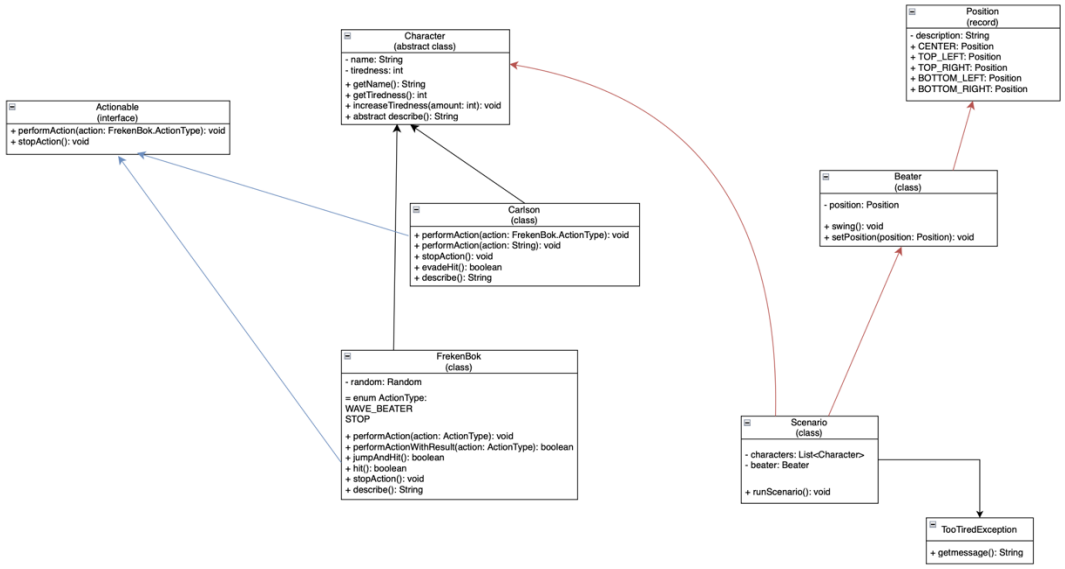
Описание предметной области, по которой должна быть построена объектная модель:

> Фрекен Бок не отставала от него, но всякий раз, когда она уже готова была его схватить, Карлсон взмывал вверх, под самый потолок. А когда фрекен Бок начинала размахивать выбивалкой, ему всегда удавалось пролететь мимо, едва ее не коснувшись. Фрекен Бок запыхалась, но продолжала подпрыгивать, и ее большие босые ноги шлепали по паркету. Она, бедняжка, так и не успела еще обуться -- ведь весь вечер ей пришлось гонять по квартире. Она очень устала, но сдаваться не собиралась.

Этапы выполнения работы:

1. Получить вариант
2. Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
3. Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
4. Согласовать диаграмму классов и сценарий с преподавателем;
5. Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
6. Продемонстрировать выполнение программы на сервере `helios`.
7. Ответить на контрольные вопросы и выполнить дополнительное задание.

Диаграмма классов объектной модели.



Исходный код программы.
Scenario

```
package carlson;

import java.util.ArrayList;
import java.util.Random;

public class Scenario {
    public static void main(String[] args) {

        ArrayList<Character> characters = new ArrayList<>();
        characters.add(new Carlson());
        characters.add(new FrekenBok());

        Beater beater = new Beater(Position.CENTER);

        System.out.println("Начало сценария");

        boolean fatigueMessageShown = false;

        Position[] positions = {
                Position.CENTER,
                Position.TOP_LEFT,
                Position.TOP_RIGHT,
                Position.BOTTOM_LEFT,
                Position.BOTTOM_RIGHT
        };
        Random random = new Random();

        try {
            while (true) {
                Carlson carlson = (Carlson) characters.get(0);
                carlson.performAction("fly");

                FrekenBok frekenBok = (FrekenBok) characters.get(1);

                if (frekenBok.getTiredness() > 30 && !fatigueMessageShown) {
                    System.out.println("Фрекен Бок запыхалась, но продолжает
размахивать выбивалкой.");
                    fatigueMessageShown = true;
                }

                Position newPosition =
positions[random.nextInt(positions.length)];
                beater.setPosition(newPosition);

                beater.swing();

                boolean attackHit =
frekenBok.performActionWithResult(FrekenBok.ActionType.WAVE_BEATER);

                if (attackHit) {
                    boolean evaded = carlson.evadeHit();
                    System.out.println(evaded ? "Карлсон увернулся от
выбивалки." : "Выбивалка задела Карлсона.");
                    if (!evaded) {
                        carlson.increaseTiredness(10);
                    }
                } else {
                    System.out.println("Фрекен Бок промахивается.");
                }
```

```
                System.out.println();
            }
        } catch (TooTiredException e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Конец.");
    }
}
```

Character

```
package carlson;

import java.util.Objects;

public abstract class Character {
    private String name;
    private int tiredness;

    public Character(String name) {
        this.name = name;
        this.tiredness = 0;
    }

    public String getName() {
        return name;
    }

    public int getTiredness() {
        return tiredness;
    }

    public void increaseTiredness(int amount) throws TooTiredException {
        tiredness += amount;
        if (tiredness >= 50) {
            throw new TooTiredException(name + " устал и больше не может
продолжать.");
        }
    }

    public abstract String describe();

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Character character = (Character) o;
        return Objects.equals(name, character.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }

    @Override
    public String toString() {
        return "Character{name='" + name + "', tiredness=" + tiredness + "}";
    }
}
```

FrekenBok

```
package carlson;

import java.util.Objects;
import java.util.Random;

public class FrekenBok extends Character implements Actionable {

    private final Random random = new Random();

    public enum ActionType {
        WAVE_BEATER,
        STOP
    }

    public FrekenBok() {
        super("Фрекен Бок");
    }

    public boolean jumpAndHit() throws TooTiredException {
        increaseTiredness(10);
        System.out.println(getName() + " подпрыгивает и бьет выбивалкой.
Усталость: " + getTiredness());
        return random.nextDouble() < 0.7;
    }

    public boolean hit() throws TooTiredException {
        increaseTiredness(5);
        System.out.println(getName() + " бьет выбивалкой. Усталость: " +
getTiredness());
        return random.nextDouble() < 0.4;
    }

    public boolean performActionWithResult(ActionType action) throws
TooTiredException {
        if (action == ActionType.WAVE_BEATER) {
            return random.nextBoolean() ? jumpAndHit() : hit();
        } else if (action == ActionType.STOP) {
            stopAction();
        }
        return false;
    }

    @Override
    public void performAction(ActionType action) {
        try {
            performActionWithResult(action);
        } catch (TooTiredException e) {
            System.out.println(e.getMessage());
        }
    }

    @Override
    public void stopAction() {
        System.out.println(getName() + " остановилась, чтобы отдышаться.");
    }

    @Override
    public String describe() {
        return getName() + " — хозяйка дома, которая пытается поймать
Карлсона.";
    }
```

```java
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof FrekenBok)) return false;
        if (!super.equals(o)) return false;
        return true;
    }

    @Override
    public int hashCode() {
        return Objects.hash(super.hashCode());
    }

    @Override
    public String toString() {
        return "FrekenBok{" + super.toString() + "}";
    }
}
```

Carlson

```
package carlson;

import java.util.Objects;

public class Carlson extends Character implements Actionable {

    public Carlson() {
        super("Карлсон");
    }

    @Override
    public void performAction(FrekenBok.ActionType action) {
        if (action == FrekenBok.ActionType.WAVE_BEATER) {
            System.out.println(getName() + " не может размахивать
выбивалкой.");
        } else {
            System.out.println(getName() + " выполняет действие.");
        }
    }

    @Override
    public void stopAction() {
        System.out.println(getName() + " останавливается.");
    }

    public void performAction(String action) throws TooTiredException {
        if ("fly".equals(action)) {
            increaseTiredness(10);
            System.out.println(getName() + " летает. Усталость: " +
getTiredness());
        } else {
            System.out.println(getName() + " не может выполнить действие: " +
action);
        }
    }

    public boolean evadeHit() {
        boolean evaded = Math.random() > 0.5;
        System.out.println(evaded ? getName() + " увернулся от выбивалки!" :
getName() + " получил удар выбивалкой!");
        return evaded;
    }

    @Override
    public String describe() {
        return getName() + " — озорной персонаж, который живет на крыше.";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Carlson)) return false;
        if (!super.equals(o)) return false;
        return true;
    }

    @Override
    public int hashCode() {
        return Objects.hash(super.hashCode());
    }
```

```
    @Override
    public String toString() {
        return "Carlson{" + super.toString() + "}";
    }
}
```

Beater

```
package carlson;

public class Beater {
    private Position position;

    public Beater(Position position) {
        this.position = position;
    }

    public void swing() {
        System.out.println("Выбивалка размахивается " +
position.description());
    }

    public void setPosition(Position position) {
        this.position = position;
    }
}
```

Position

```
package carlson;

public record Position(String description) {
    public static final Position CENTER = new Position("в центре комнаты");
    public static final Position TOP_LEFT = new Position("в верхнем левом
углу");
    public static final Position TOP_RIGHT = new Position("в верхнем правом
углу");
    public static final Position BOTTOM_LEFT = new Position("в нижнем левом
углу");
    public static final Position BOTTOM_RIGHT = new Position("в нижнем правом
углу");
}
```

Actionable

```
package carlson;

public interface Actionable {
    void performAction(FrekenBok.ActionType action);
    void stopAction();
}
```

TooTiredException

```
package carlson;

public class TooTiredException extends Exception {
    public TooTiredException(String message) {
        super(message);
    }
}
```

Результат работы программы.

```
Начало сценария
Карлсон летает. Усталость: 5
Выбивалка размахивается в нижнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 5
Карлсон увернулся от выбивалки.

Карлсон летает. Усталость: 10
Выбивалка размахивается в нижнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 10
Карлсон увернулся от выбивалки.

Карлсон летает. Усталость: 15
Выбивалка размахивается в нижнем правом углу
Фрекен Бок бьет выбивалкой. Усталость: 15
Фрекен Бок промахивается.

Карлсон летает. Усталость: 20
Выбивалка размахивается в нижнем левом углу
Фрекен Бок подпрыгивает и бьет выбивалкой. Усталость: 25
Выбивалка задела Карлсона.

Карлсон летает. Усталость: 35
Выбивалка размахивается в нижнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 30
Выбивалка задела Карлсона.

Карлсон летает. Усталость: 50
Выбивалка размахивается в нижнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 35
Выбивалка задела Карлсона.

Карлсон устал и больше не может летать.
Конец.
```

```
Начало сценария
Карлсон летает. Усталость: 5
Выбивалка размахивается в нижнем левом углу
Фрекен Бок подпрыгивает и бьет выбивалкой. Усталость: 10
Выбивалка задела Карлсона.

Карлсон летает. Усталость: 20
Выбивалка размахивается в верхнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 15
Фрекен Бок промахивается.

Карлсон летает. Усталость: 25
Выбивалка размахивается в нижнем левом углу
Фрекен Бок подпрыгивает и бьет выбивалкой. Усталость: 25
Фрекен Бок промахивается.

Карлсон летает. Усталость: 30
Выбивалка размахивается в верхнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 30
Фрекен Бок промахивается.

Карлсон летает. Усталость: 35
Выбивалка размахивается в нижнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 35
Фрекен Бок промахивается.

Карлсон летает. Усталость: 40
Фрекен Бок запыхалась, но продолжает размахивать выбивалкой.
Выбивалка размахивается в верхнем левом углу
Фрекен Бок бьет выбивалкой. Усталость: 40
Фрекен Бок промахивается.

Карлсон летает. Усталость: 45
Выбивалка размахивается в нижнем правом углу
Фрекен Бок подпрыгивает и бьет выбивалкой. Усталость: 50
Карлсон увернулся от выбивалки.

Фрекен Бок устала и больше не может размахивать выбивалкой.
Конец.
```

Выводы по работе.

В ходе работы я разработала объектную модель, отражающую взаимодействие персонажей и предметов. Реализованы основные классы, включая абстрактный базовый класс и интерфейс, соблюдены принципы ООП: инкапсуляция, наследование, полиморфизм. Сценарий взаимодействий учитывает усталость персонажей, элементы случайности и обработку исключений.